NetTrace

AI-Powered Network Attack Path Reconstruction

1. Problem Statement

Modern enterprise networks generate millions of packets per day. When a security breach occurs, forensic analysts are left with raw packet capture (.pcap) files containing thousands of flows, with no immediate way to determine which connections were malicious, which host initiated the attack, or how lateral movement progressed through the network.

Existing Intrusion Detection Systems (IDS) such as Snort or Suricata can flag individual suspicious packets using rule-based signatures, but they cannot answer the higher-level forensic question: what was the complete attack path — from the initial entry point, through every intermediate pivot, to the final target?

NetTrace addresses this gap. Given a raw .pcap file, the system automatically identifies the attacker, reconstructs every attack path through the network, and highlights the single most critical node that, if blocked, would disrupt all attack routes simultaneously. This transforms hours of manual forensic analysis into a pipeline that completes in seconds.

2. System Architecture

NetTrace is structured as a six-stage pipeline. Each stage is implemented in a dedicated Python module with a single, well-defined responsibility. The AI module handles unstructured data; the classical algorithms handle all core logic.

Stage	Module	Responsibility	Technology
1	**pcap_parser.py**	PCAP parsing	Scapy — reads IPv4 and IPv6 packets, groups into flows
2	**ai_classifier.py**	AI risk classification	Gemini API — assigns label and risk score (0–100) per flow
3	**graph_builder.py**	Graph construction	Builds weighted directed graph; de-duplicates edges by highest risk
4	**dijkstra.py**	Attack path finding	Custom min-heap Dijkstra — finds all high-risk paths from attacker
5	**greedy.py**	Critical node selection	Greedy scoring — finds the node whose removal disrupts most paths
6	**gui.py**	Visualization	Tkinter + NetworkX + Matplotlib — interactive graph and report panel

2.1 AI ↔ Algorithm Communication

The AI module (Stage 2) outputs a risk_score (integer 0–100) and a label (string) for each network flow. These scores are the only bridge between the AI and the algorithm layers. The graph builder converts each risk score into an edge weight using the formula:

$$\texttt{edge\_weight} = ((100 - \texttt{risk\_score}) / 100)^2$$

This squaring is intentional: it amplifies the difference between high-risk and medium-risk edges, making Dijkstra more decisively favor the most dangerous path over ambiguous middle-ground paths. A risk score of 91/100 produces a weight of 0.0081, while a score of 50/100 produces 0.25 — a 30x difference that strongly guides path selection.

3. Algorithmic Logic

3.1 Graph Representation

The network is modeled as a weighted directed graph G = (V, E) where:

- V (vertices) = all unique IP addresses observed in the PCAP file
- E (edges) = directed connections between IPs, one edge per (src, dst) pair
- w(e) = edge weight derived from the AI risk score as described above

When multiple flows share the same (src, dst) pair (e.g., port 445 and port 135 both from the same source to the same destination), only the highest-risk flow is retained. This prevents Dijkstra from accidentally routing through a low-risk duplicate when a higher-risk edge exists between the same nodes.

3.2 Dijkstra's Algorithm — Custom Min-Heap Implementation

Dijkstra's algorithm was implemented from scratch using Python's heapq module as the underlying min-heap. The standard library's heapq provides O(log n) push and pop operations, giving the overall algorithm a time complexity of O((V + E) log V).

The algorithm operates as follows:

- Initialize all distances to infinity; set distance of the attacker node to 0
- Push (0.0, attacker) onto the min-heap
- At each step, pop the node with the lowest cumulative weight
- For each neighbor, compute new_cost = current_cost + edge_weight
- If new_cost is less than the recorded distance, update and push to heap
- Track previous[node] and edge_used[node] to enable path reconstruction

A single Dijkstra run from the identified attacker computes the minimum-cost (= maximum-risk) path to every reachable node simultaneously. Paths to all identified victims are then reconstructed by tracing backwards through the previous[] map.

3.3 Attacker Identification Heuristic

The attacker is identified without any hardcoded IP ranges. The algorithm scores every routable IP by its total outgoing risk (sum of risk scores of all outgoing flows). External IPs (non-RFC-1918) are prioritized as attacker candidates. If no external IP exists, the algorithm falls back to internal IPs with zero incoming risk but positive outgoing risk — indicating a node that only sends attacks but never receives them.

3.4 Greedy Critical Node Algorithm

The Greedy algorithm identifies the single node whose removal would disrupt the maximum number of attack paths. For each candidate node (excluding the attacker and leaf victims), the algorithm counts how many of the reconstructed attack paths pass through it. The node with the highest count is selected as the Critical Node. This is a Greedy approximation of the NP-hard minimum vertex cut problem, running in O(P × N) where P is the number of paths and N is the number of nodes.

4. AI Integration Details

4.1 Model Used

NetTrace uses Google Gemini (via the google-genai SDK) for connection classification. The model receives batches of up to 15 raw connection descriptions and returns a structured JSON array with a label and risk_score for each connection. Gemini was selected for its generous free-tier quota, making the project reproducible without API costs.

4.2 Prompt Design

The system prompt instructs the model to act as a cybersecurity expert and base its scoring on: port number, protocol, packet count, connection duration, TCP flags, and known attack patterns. The model is instructed to return only a raw JSON array with no explanation or markdown formatting. A regex-based parser strips any accidental fences before JSON parsing.

4.3 What the AI Handles vs. What the Algorithm Handles

AI (Gemini)	Classical Algorithm (Dijkstra + Greedy)
Reads raw connection text (unstructured)	Receives clean risk scores (structured numbers)
Assigns contextual risk scores 0–100	Builds weighted graph from scores
Labels attack type (brute_force_ssh, etc.)	Finds minimum-weight (maximum-risk) paths
Handles ambiguity and context naturally	Guarantees optimal path with O((V+E) log V)

5. Complexity Analysis

Component	Complexity	Notes
PCAP Parsing	O(P)	P = total packets in file
AI Classification	O(F / 15)	F = flows; batched in groups of 15
Graph Construction	O(F)	One pass over all flows
Dijkstra (min-heap)	O((V+E) log V)	V = IPs, E = unique flows; optimal for sparse graphs
Greedy Critical Node	O(Pa × N)	Pa = attack paths, N = nodes per path